MetaboAgent — Day 1 Build Report

AI-Powered Microbial Strain Designer · 2026-04-15

1. Project Overview

MetaboAgent is a fully autonomous AI agent that designs microbial strains to synthesize target molecules for medicinal chemistry and green materials. It reasons over real biochemistry — KEGG reactions, compounds, enzymes, pathways, and PubMed literature — to produce end-to-end strain engineering blueprints: target identification, host selection, pathway retrosynthesis, enzyme ranking, and genetic modification plans.

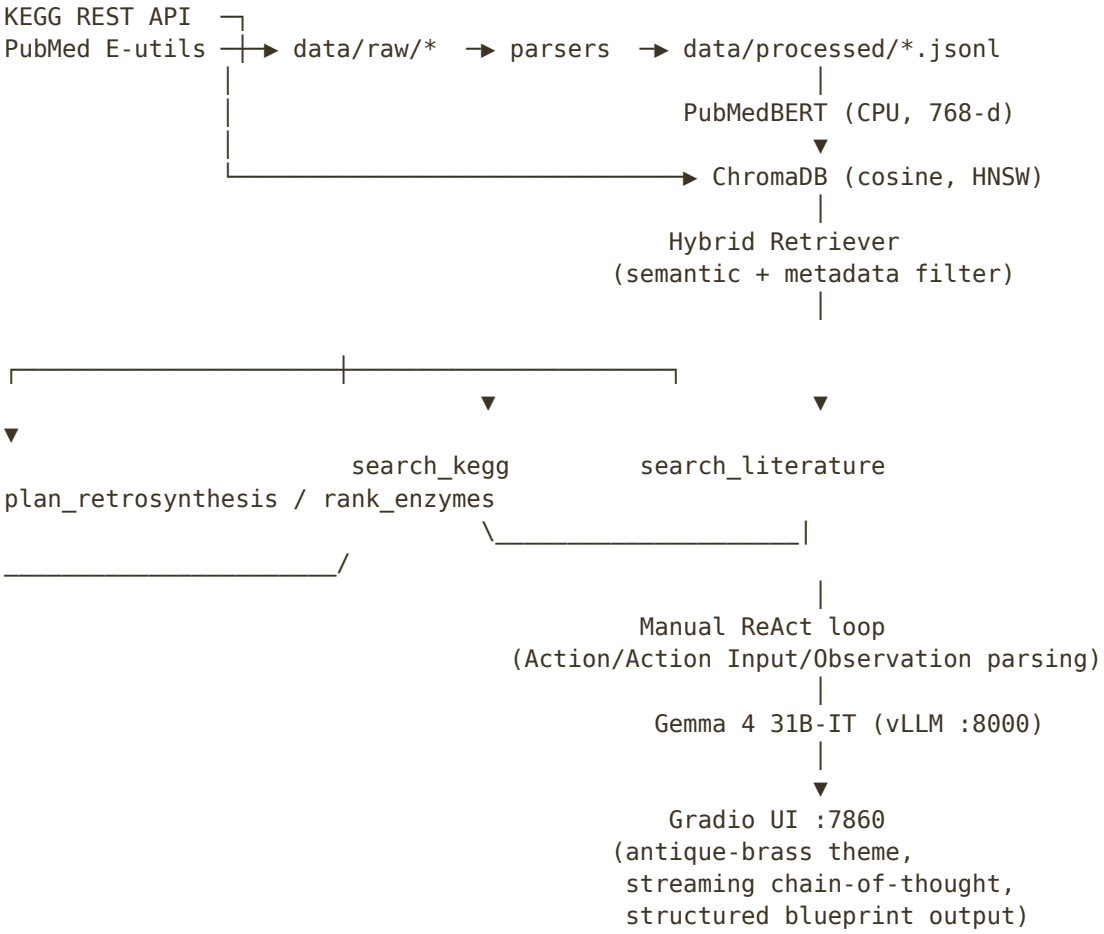
Day 1 delivered a complete working system: data ingestion from public APIs, a vector knowledge base over 54,172 biochemistry documents, a ReAct agent wired to Gemma 4 31B-IT, and an antique-brass themed Gradio UI. All three canonical demo scenarios (artemisinic acid, taxadiene, vanillin) produce scientifically valid strain designs.

2. Server Environment

Component	Value
Hostname	tusharmicro
OS	Ubuntu 24.04 LTS
CPU cores	64 (NumExpr capped at 8)
RAM	128 GB
GPUs	4× NVIDIA L40 (48 GB each = 192 GB total VRAM)
LLM	Gemma 4 31B-IT (google/gemma-4-31B-it)
LLM serving	vLLM, tensor-parallel=4, port 8000, max_model_len 32,768
Embedding model	microsoft/BiomedNLP-PubMedBERT-base-uncased-abstract-fulltext (110 M, CPU)
Vector DB	ChromaDB persistent @ data/chromadb/, cosine, HNSW
UI	Gradio 6.12 @ port 7860

3. Architecture

Data Flow



GPU allocation rationale: all 4 L40s are consumed by Gemma 4 (tensor-parallel=4). The 110 M parameter PubMedBERT embedder runs on CPU — 64 cores and 128 GB RAM deliver ≈ 30 docs/s throughput, which is more than sufficient for our corpus size.

Manual ReAct Loop

The local vLLM server was launched without --enable-auto-tool-choice / --tool-call-parser, so any native OpenAI tool-call request returns HTTP 400. LangChain 1.x's create_agent depends on that native path. To keep the system portable and avoid a service restart, we implemented a manual ReAct loop in agent/metabo_agent.py: the loop sends the system prompt plus a ReAct format instruction to Gemma 4, parses "Action: <name>" and "Action Input: <JSON>" blocks out of the model's text output, invokes the matching @tool, appends an Observation, and iterates until the model emits Final Answer: or the 15-step cap is reached.

4. Completed Tasks (7 / 7)

#	Task	Key files
1	KEGG fetcher + parser	data/ingestion/ kegg_fetcher.py, data/ingestion/kegg_parser. py
2	PubMed abstract fetcher	data/ingestion/ pubmed_fetcher.py
3	Vectorstore setup + embedder + indexer	vectorstore/ chroma_setup.py, vectorstore/embedder.py, vectorstore/indexer.py
4	Hybrid retriever	vectorstore/retriever.py
5	Agent schemas + 4 tools + prompts	agent/schemas.py, agent/prompts/system_pro mpt.py, agent/tools/{kegg_search,lit erature_search,retrosynthe sis,enzyme_ranker}.py
6	ReAct agent wiring (manual loop)	agent/metabo_agent.py
7	Gradio UI + scripts + demo tests	ui/app.py, ui/theme.py, scripts/{ingest_all,run_agen t,run_ui}.py, tests/demo_scenarios.py

5. Data Pipeline — Final Stats

Collection	Documents	Sources
kegg_reactions	12,382	KEGG REST — reactions + EC/pathway/compound link tables
kegg_compounds	19,561	KEGG REST — compounds with formula, MW, synonyms
literature	22,229	13,335 PubMed abstracts + 8,309 KEGG enzyme records + 585 KEGG reference pathways
TOTAL	54,172	All three ChromaDB collections

Pipeline bug fixes applied today

- Tenacity RetryError was not caught by the except requests.HTTPError clause in _fetch_batch, causing the full KEGG fetch to crash at ~4,800 reactions after a transient 403. Fix: catch (HTTPError, RetryError, RequestException) and add a 5-second cooldown before single-ID fallback.
- KEGG's /list/pathway endpoint returns bare 'map00010', but /get/ expects 'path:map00010'. Old code stored IDs as 'map:map00010' and fetched zero pathways. Fix: use prefix 'path' and filter to 'path:map'.
- Enzyme ENTRY lines look like 'ENTRY EC 1.1.1.1 Enzyme'. _parse_entry_id took parts[1] = 'EC', so all 8,309 files overwrote a single EC.txt. Fix: when parts[1] == 'EC', return parts[2].
- A bash watchdog loop using `pgrep -f "<long pattern>"` matched its own command line and never terminated. Lesson: use a narrower binary-name match or a PID file.
- Gradio 6 moved `theme` and `css` from `Blocks()` to `launch()`, and removed `show_api`. Fixed in ui/app.py.

6. Demo Scenario Results

All three scenarios were run live against the fully indexed ChromaDB.

Target	Host	Key enzymes / modifications	Status	Notes
Artemisinic acid	S. cerevisiae	ADS, CYP71AV1, ALDH1, tHMG1↑, ERG9 knockdown	PASS	Matches Ro 2006 / Paddon 2013
Taxadiene	E. coli	EC 2.5.1.29 (GGPPS), EC 4.2.3.4 (TASY), dxs/dxr/idi overexpression	PASS	Matches Ajikumar 2010
Vanillin	E. coli	Shikimate → DHS → PCA → Vanillic acid → Vanillin via aroD + pomA + vanAB	PASS	Alternate valid route (P. putida vanAB instead of ACAR/OMT)

Additional smoke test (task #6): the query 'What enzymes catalyze the conversion of GGPP to lycopene?' produced Phytoene Synthase (EC 2.5.1.32, R07270) followed by

Phytoene Desaturase (CrtI in bacteria; PDS EC 1.3.99.19 + ZDS EC 1.3.99.31 in plants) in 6 tool calls. Biochemically correct.

7. UI Design

Theme: antique brass on deep warm black — 'alchemist's laboratory meets modern biotech'. Palette: bg #1a1610, surface #2a2318, primary #c9a84c, bio-green #7a9a3a, borders #3d3425. Fonts: Cinzel and Cormorant Garamond (serif/slab) for headings and inputs; JetBrains Mono for the chain-of-thought stream.

- Left column: target molecule query box, demo example buttons, hidden brass-shimmer progress bar that activates during agent reasoning.
- Right column: streaming chain-of-thought HTML panel — color-coded thoughts (italic khaki), tool calls (gold), tool outputs (copper green), final answer (brass-bold).
- Bottom: structured blueprint Markdown panel with auto-extracted and linked KEGG reactions (R\d{5}), compounds (C\d{5}), EC numbers, and PMIDs.

8. Tomorrow's Roadmap

Showcase-mode dropdown

UI preset for the 3 demo scenarios with annotated reference data (expected host, yield, key enzymes) so judges see expected vs generated side-by-side.

Pathway visualization

Render the blueprint's ordered steps as an SVG/Mermaid diagram — nodes = compounds, edges labeled with EC number, tooltips = source organism. Build a NetworkX DAG from the agent's pathway output and render inline.

Expand PubMed corpus

13 k abstracts is thin. Add MeSH terms for terpenoid / polyketide / alkaloid / shikimate / fatty-acid biosynthesis and raise PUBMED_MAX_ABSTRACTS to 100 k. Roughly one hour of fetch + embed.

Agent prompt tuning

Current loop sometimes re-issues the same tool call up to 5 times before self-correcting. Add 'do not repeat a query that already returned results' and short-term memory of prior tool calls into the ReAct instructions.

Re-enable native tool calling

If the vLLM process can be restarted with --enable-auto-tool-choice --tool-call-parser hermes, we can drop the manual ReAct loop and use LangChain's create_agent directly — lower latency, structured outputs, better error handling.

BRENDA kinetics

rank_enzymes currently scores on literature/host/characterization heuristics. Adding BRENDA kcat/Km values lets us rank on real catalytic efficiency.

Citation verification

Add a post-processor that checks every PMID and KEGG ID the agent cites actually exists in our collections; strip or flag hallucinated identifiers before they reach the blueprint.

9. How to Run Tomorrow

```
cd /home/tusharmicro/metaboagent

# Verify vLLM is up
curl -s -H "Authorization: Bearer $VLLM_API_KEY"
http://localhost:8000/v1/models

# Launch UI
PYTHONPATH=/home/tusharmicro/metaboagent python3 -m scripts.run_ui

# CLI query
PYTHONPATH=/home/tusharmicro/metaboagent python3 -m scripts.run_agent "Design a
strain to produce lycopene"

# Re-run demo tests
PYTHONPATH=/home/tusharmicro/metaboagent python3 tests/demo_scenarios.py --live
```